Universal Auto-Healing Feature Complete Testing Guide

Covers all 3 implementation phases: Phase 1 Core Engine & CLI Phase 2 AI Healer & Dashboard Phase 3 Settings, Fingerprints & Polish

Table of Contents

1. Prerequisites & Setup
2. Part 1: CLI Testing (REPL Commands)
3. Part 2: API Testing (HTTP Bridge)
4. Part 3: UI Testing (Browser)
5. Part 4: WebSocket Real-Time Testing
6. Part 5: Error Handling Testing
7. Part 6: End-to-End Integration Test
8. Quick Reference Tables

Prerequisites & Setup

```
# 1. Build everything
pnpm run build:ui && pnpm build

# 2. Start the qabot UI server (port 3700)
node bin/qabot.js /buildUI

# 3. Open browser
# Navigate to http://localhost:3700
```

Note: The healing SQLite database is stored at `~/.qabot/healing.db`. If you want a clean test, delete this file before starting. On Windows the path is `C:\Users\{username}\.qabot\healing.db`.

Part 1: CLI Testing (REPL Commands)

All tests below run inside the qabot REPL (interactive mode).

Test 1.1 — `/heal` (No Subcommand)

```
/heal
```

> **Expected:** Shows healing status summary — fingerprint count, event stats, success rate. If no data exists yet, shows zeros.

Test 1.2 — `/heal status`

```
/heal status
```

> **Expected:** Formatted table showing: • Fingerprints stored count • Healing events (total / healed / failed) • Success rate % • Avg confidence, avg duration • Strategy breakdown (if events exist) • Framework breakdown (if events exist) • List of injected projects (if any)

Test 1.3 — `/heal adapters`

```
/heal adapters
```

> **Expected:** Lists all 6 supported adapter combinations: 1. Playwright — TypeScript/JavaScript 2. Selenium — Java 3. Selenium — Python 4. Cypress — TypeScript 5. WebdriverIO — TypeScript 6. Selenium — C#

Test 1.4 — `/heal inject` (Auto-detect Framework)

```
# First, create a dummy Playwright project:
mkdir /tmp/test-heal-project
cd /tmp/test-heal-project
npm init -y
npm install @playwright/test
```

```
# In qabot REPL:
/heal inject /tmp/test-heal-project
```

Expected: • Auto-detects Playwright + TypeScript • Creates 2 files: `tests/helpers/healing-client.ts` and `tests/fixtures/healing.ts` • Shows integration instructions (import the fixture, use `test()` instead of default) • Shows configuration (server URL, confidence threshold, AI enabled)

Test 1.5 — Verify Injection was Recorded

```
/heal status
```

Verify: Shows the injected project in the list with path, framework: playwright, language: typescript, status: active.

Test 1.6 — `/heal report`

```
/heal report 7
/heal report 30
/heal report 90
```

Expected: Each shows analytics for the specified time window (likely empty/zero if fresh database).

Test 1.7 — `/heal remove`

```
/heal remove /tmp/test-heal-project
```

Expected: Marks the injection as removed. `/heal status` should reflect the change.

Part 2: API Testing (HTTP Bridge)

These test the healing API endpoints that external test frameworks call. Use `curl` from a terminal while qabot UI server is running on port 3700.

Test 2.1 — Store a Fingerprint (Baseline)

```
curl -s -X POST http://localhost:3700/api/heal/fingerprint \
  -H "Content-Type: application/json" \
  -d '{
    "selectorKey": "css:#login-button",
    "url": "https://example.com/login",
    "fingerprint": {
      "tagName": "button",
      "id": "login-button",
      "testId": "login-btn",
      "className": "btn btn-primary",
      "ariaLabel": "Sign In",
      "textContent": "Sign In",
      "type": "submit"
    },
    "framework": "playwright"
  }' | jq .
```

Expected: `{ "id": "...", "stored": true }`

Test 2.2 — Store Additional Fingerprints

```
# Second fingerprint (search input)
curl -s -X POST http://localhost:3700/api/heal/fingerprint \
  -H "Content-Type: application/json" \
  -d '{
    "selectorKey": "css:#search-input",
    "url": "https://example.com/search",
    "fingerprint": {
      "tagName": "input",
      "id": "search-input",
      "testId": "search-box",
      "className": "form-control search-field",
      "ariaLabel": "Search",
      "placeholder": "Search products...",
      "type": "text",
      "name": "q"
    },
```

```
    "framework": "playwright"
  }' | jq .

# Third fingerprint (Selenium framework)
curl -s -X POST http://localhost:3700/api/heal/fingerprint \
  -H "Content-Type: application/json" \
  -d '{
    "selectorKey": "css:.nav-menu .dropdown",
    "url": "https://example.com/dashboard",
    "fingerprint": {
      "tagName": "div",
      "className": "nav-menu dropdown active",
      "ariaLabel": "Navigation Menu",
      "textContent": "Settings"
    },
    "framework": "selenium"
  }' | jq .
```

Test 2.3 — Resolve a Broken Selector (Main Healing Endpoint)

```
curl -s -X POST http://localhost:3700/api/heal/resolve \
  -H "Content-Type: application/json" \
  -d '{
    "selector": "#login-btn-old",
    "selectorStrategy": "css",
    "fingerprint": {
      "tagName": "button",
      "id": "login-btn-old",
      "testId": "login-btn",
      "className": "btn btn-primary updated",
      "ariaLabel": "Sign In",
      "textContent": "Sign In",
      "type": "submit"
    },
    "pageUrl": "https://example.com/login",
    "framework": "playwright",
    "language": "typescript"
  }' | jq .
```

Expected: Returns healed selector:

```
{
  "healed": true,
  "selector": "[data-testid=\"login-btn\"]",
  "selectorStrategy": "css",
  "confidence": 0.90,
  "strategy": "fingerprint",
```

```
    "durationMs": ...,
    "aiUsed": false,
    "candidates": [...]
  }
```

Test 2.4 — Resolve Without Stored Fingerprint (Should Fail)

```
curl -s -X POST http://localhost:3700/api/heal/resolve \
  -H "Content-Type: application/json" \
  -d '{
    "selector": "#unknown-element",
    "pageUrl": "https://example.com/unknown",
    "framework": "cypress"
  }' | jq .
```

Expected: `{ "healed": false, ... }` — no stored fingerprint to match against.

Test 2.5 — Report Healing Outcome (Success)

```
curl -s -X POST http://localhost:3700/api/heal/report \
  -H "Content-Type: application/json" \
  -d '{
    "selectorKey": "css:#login-btn-old",
    "url": "https://example.com/login",
    "healed": true,
    "strategy": "fingerprint",
    "confidence": 0.90,
    "framework": "playwright",
    "language": "typescript",
    "originalSelector": "#login-btn-old",
    "healedSelector": "[data-testid=\"login-btn\"]",
    "durationMs": 45
  }' | jq .
```

Expected: `{ "id": "...", "recorded": true }`

Test 2.6 — Report a Failed Healing

```
curl -s -X POST http://localhost:3700/api/heal/report \
  -H "Content-Type: application/json" \
  -d '{
```

```
    "selectorKey": "css:.dynamic-widget",
    "url": "https://example.com/dashboard",
    "healed": false,
    "framework": "selenium",
    "language": "java",
    "originalSelector": ".dynamic-widget",
    "durationMs": 250
  }' | jq .
```

Test 2.7 — Query Analytics

```
curl -s http://localhost:3700/api/heal/analytics?days=30 | jq .
```

> **Expected:** Full analytics object with `totalEvents`, `totalHealed`, `totalFailed`, `overallSuccessRate`, `strategyBreakdown`, `frameworkBreakdown` (playwright + selenium), `timeline`, and `topFailures`.

Test 2.8 — Query Events with Filters

```
# All events
curl -s "http://localhost:3700/api/heal/events?days=30" | jq .

# Filter by framework
curl -s "http://localhost:3700/api/heal/events?framework=playwright&d

# Filter by success
curl -s "http://localhost:3700/api/heal/events?success=true&days=30"

# With pagination
curl -s "http://localhost:3700/api/heal/events?limit=5&offset=0" | jq
```

> **Expected:** Returns `{ events: [...], total: N }` with appropriate filtering.

Test 2.9 — List Adapters

```
curl -s http://localhost:3700/api/heal/adapters | jq .
```

> **Expected:** 6 adapters: Playwright (TS), Selenium (Java), Selenium (Python), Cypress (TS), WebdriverIO (TS), Selenium (C#)

Test 2.10 — Fingerprint CRUD

```
# List all fingerprints
curl -s "http://localhost:3700/api/healing/fingerprints" | jq .

# Search fingerprints
curl -s "http://localhost:3700/api/healing/fingerprints?search=login"

# Paginate
curl -s "http://localhost:3700/api/healing/fingerprints?limit=10&offs

# Delete a fingerprint (use actual ID from the list above)
curl -s -X DELETE "http://localhost:3700/api/healing/fingerprints/<ID
```

Test 2.11 — Healing Config API

```
# Read config
curl -s http://localhost:3700/api/healing/config | jq .

# Update config
curl -s -X PUT http://localhost:3700/api/healing/config \
  -H "Content-Type: application/json" \
  -d '{ "confidenceThreshold": 0.8, "aiEnabled": false }' | jq .

# Verify update persisted
curl -s http://localhost:3700/api/healing/config | jq .
```

> **Expected:** Config reads back with updated values: `confidenceThreshold: 0.8`, `aiEnabled: false`

Test 2.12 — Healing Stats

```
curl -s http://localhost:3700/api/healing/stats | jq .
```

> **Expected:** `{ "total": N, "successCount": N, "failureCount": N, "successRate": N, "available": true }`

Part 3: UI Testing (Browser)

Open `http://localhost:3700` in your browser. If auth is configured, log in first.

Test 3.1 — Main Dashboard → HealingCard

1. Navigate to `/` (Dashboard)
2. **Verify:** A "Self-Healing" card appears in the bottom section
3. **Verify:** Shows "Elements tracked" count and "Success rate" percentage
4. **Verify:** Card is clickable — clicking navigates to `/healing`

> **Note:** If no healing data exists, the card may be hidden (returns null when `available: false`).

Test 3.2 — Healing Dashboard → Overview Tab

1. Navigate to `/healing` (click "Healing" in the sidebar or the dashboard card)
2. **Verify:** Page title "Self-Healing Dashboard" with Heart icon
3. **Verify:** Day filter dropdown (7 / 30 / 90 days) in top-right corner
4. **Verify:** 6 overview stat cards at the top:
   - Total Events (blue Activity icon)
   - Success Rate (color-coded: green ≥80%, amber ≥50%, red <50%)
   - Avg Confidence (brand color)
   - Avg Duration (cyan Clock icon)
   - AI Usage % (violet Sparkles icon)
   - Active Projects (pink FolderOpen icon)
5. **Verify:** 5 tabs visible: **Overview** | **Events** | **Fingerprints** | **Projects** | **Adapters**
6. **Verify:** Overview tab shows (when data exists):
   - **Healing Trend** chart (stacked area: green healed / red failed + purple success % line)
   - **Strategy Breakdown** bar chart (color-coded per strategy)
   - **Framework Breakdown** horizontal bar chart
   - **Top Failures** table (selector, URL, failure count, last seen)
7. **Test day filter:** Change from 30 → 7 days → verify data updates
8. **Test empty state:** With no data, charts show "No healing data yet" messages

Test 3.3 — Healing Dashboard → Events Tab

1. Click "Events" tab
2. **Verify:** Search bar, Framework dropdown, Result dropdown (All/Healed/Failed), event count
3. **Verify:** Table columns: Status | Selector | Healed To | Strategy | Framework | Confidence | Duration | Time
4. **Verify:** Status icons: green checkmark for healed, red X for failed
5. **Verify:** AI-powered events show purple Sparkles icon next to strategy
6. **Verify:** Confidence is color-coded (green ≥0.8, amber ≥0.5, red <0.5)
7. **Test search:** Type a selector name → table filters in real-time
8. **Test framework filter:** Select "playwright" → only Playwright events shown
9. **Test result filter:** Select "Healed" → only successful heals shown
10. **Test pagination:** If >25 events, verify prev/next buttons and "Showing X–Y of Z" text

Test 3.4 — Events Tab → Detail Panel

1. On the Events tab, **click any event row**
2. **Verify:** A 380px side panel opens on the right
3. **Verify panel contents:**
    - Header with Healed/Failed badge + AI badge (if applicable) + X close button
    - **Selector Change:** Original selector (red box) → arrow → Healed selector (green box)
    - **Strategy:** Name + human-readable description
    - **Confidence:** Large % number + High/Medium/Low label + progress bar
    - **Details grid:** URL, Framework, Language, Duration, Timestamp
4. **Verify:** Selected row is highlighted in the table (left blue border)
5. **Click another row:** Panel updates to show new event details
6. **Click X button:** Panel closes, row highlight removed

Test 3.5 — Healing Dashboard → Fingerprints Tab

1. Click "Fingerprints" tab
2. **Verify:** Search bar at top
3. **Verify:** Table columns: Selector Key | URL | Success | Failures | Updated | Delete
4. **Verify:** Fingerprints stored via API (Tests 2.1–2.2) are visible
5. **Test search:** Type "login" → filters to matching fingerprints
6. **Test delete:**
    - Click trash icon on a fingerprint row

    - Verify: Confirmation appears ("Confirm" / "Cancel")
    - Click "Cancel" → nothing happens
    - Click trash again → click "Confirm" → fingerprint is deleted
    - Verify: Fingerprint disappears from list
7. **Test pagination:** If >25 fingerprints, verify page controls work
8. **Test empty state:** If all deleted, shows "No fingerprints found" with Fingerprint icon

Test 3.6 — Healing Dashboard → Projects Tab

1. Click "Projects" tab
2. **Verify:** Cards for injected projects (from `/heal inject` or API)
3. Each card should show:
    - Project name + full path
    - Status badge (active=green, disabled=amber, removed=red)
    - Framework + Language grid
    - Confidence threshold + AI enabled/disabled
    - Generated files list (up to 4 shown, "+N more" if more)
    - Healing server URL
    - Injection timestamp
4. **Test remove:** Click "Remove" → toast: "Injection removed successfully"
5. **Test empty state:** Shows "No injected projects" with instructions

Test 3.7 — Healing Dashboard → Adapters Tab

1. Click "Adapters" tab
2. **Verify:** 6 adapter cards in a 3-column grid
3. Cards: Playwright (rose), Selenium Java (amber), Selenium Python (amber), Cypress (cyan), WebdriverIO (violet), Selenium C# (amber)
4. Each card shows: Display name, Framework tag, Language tag, Puzzle icon

Test 3.8 — Settings Page → Healing Section

1. Navigate to `/settings`
2. **Verify:** "Self-Healing" card exists between "Appearance" and "Cloud Grid Providers"
3. **Verify controls:**
    - Enable self-healing checkbox (default: checked)
    - Confidence Threshold slider (0.10–1.00, default: 0.70)

    - Enable AI-powered healing checkbox (default: checked)
    - Data Retention dropdown (30/60/90/180 days, default: 90)
4. **Test slider:** Drag confidence threshold → value label updates in real-time
5. **Test save:**
    - Change confidence to 0.85, uncheck AI, change retention to 60 days
    - Click "Save Healing Settings" → success toast appears
    - Refresh page → settings persist
6. **Verify via API:** `curl http://localhost:3700/api/healing/config` shows updated values

Part 4: WebSocket Real-Time Testing

Test 4.1 — Real-Time Event Updates

1. Open `/healing` → Events tab in the browser
2. Note the current event count
3. In a terminal, trigger a healing resolve:

```
curl -s -X POST http://localhost:3700/api/heal/resolve \
  -H "Content-Type: application/json" \
  -d '{
    "selector": "#realtime-test-btn",
    "fingerprint": {
      "tagName": "button",
      "testId": "realtime-btn",
      "textContent": "Click Me"
    },
    "pageUrl": "https://example.com/realtime",
    "framework": "cypress",
    "language": "typescript"
  }'
```

> **Verify (no page refresh needed):**
>
> - Events table updates automatically with new event
> - Overview cards update (Total Events count increases)
> - If on Overview tab, trend chart data refreshes

Test 4.2 — Real-Time Analytics Update

1. Open `/healing` → Overview tab
2. Run several resolve + report curl commands with different frameworks
3. **Verify:** Strategy breakdown chart updates in real-time
4. **Verify:** Framework breakdown shows new frameworks
5. **Verify:** Timeline chart reflects new data points

Part 5: Error Handling Testing

Test 5.1 — Network Error States

1. Navigate to `/healing`
2. Stop the qabot server (Ctrl+C in the terminal)
3. Wait ~30 seconds for React Query auto-refetch to fire
4. **Verify:** All healing components show red error cards:

Component	Expected Error Message
HealingOverviewCards	"Failed to load healing overview"
HealingTrendChart	"Failed to load trend data"
StrategyBreakdownChart	"Failed to load strategy data"
FrameworkBreakdownChart	"Failed to load framework data"
TopFailuresTable	"Failed to load failure data"
HealingEventsTable	"Failed to load events"
InjectedProjectsPanel	"Failed to load projects"
AdaptersList	"Failed to load adapters"

> **Verify:** Each error card shows AlertCircle icon + error message + technical detail text.
> **Restart server** → components auto-recover on next refetch cycle.

Test 5.2 — Fingerprint Browser Error State

1. Navigate to `/healing` → Fingerprints tab
2. Stop the server
3. **Verify:** Red error card: "Failed to load fingerprints"
4. Restart server → data reappears

Test 5.3 — InjectedProjectsPanel Toast on Error

1. Navigate to `/healing` → Projects tab
2. Click "Remove" on a project when server is down
3. **Verify:** Error toast appears ("Remove failed: ...")

Part 6: End-to-End Integration Test

This is the full realistic workflow: inject healing → build baselines → break selectors → verify auto-heal.

Step 1: Create a Test Project

```
mkdir /tmp/e2e-heal-test && cd /tmp/e2e-heal-test
npm init -y
npm install @playwright/test
npx playwright install chromium
```

Step 2: Inject Healing via CLI

```
# In qabot REPL:
/heal inject /tmp/e2e-heal-test
```

> **Verify:** Two files created under `tests/helpers/` and `tests/fixtures/` .

Step 3: Verify in UI

1. Go to `http://localhost:3700/healing` → Projects tab
2. **Verify:** `/tmp/e2e-heal-test` appears as active injection

Step 4: Store Baselines via API

```
# Simulate what the injected code does on a passing test
curl -s -X POST http://localhost:3700/api/heal/fingerprint \
  -H "Content-Type: application/json" \
  -d '{
    "selectorKey": "css:h1",
    "url": "https://example.com",
    "fingerprint": {
      "tagName": "h1",
      "textContent": "Example Domain",
      "className": ""
    },
    "framework": "playwright"
  }'
```

Step 5: Simulate a Broken Selector

```
curl -s -X POST http://localhost:3700/api/heal/resolve \
  -H "Content-Type: application/json" \
  -d '{
    "selector": "h1.old-heading",
    "pageUrl": "https://example.com",
    "framework": "playwright",
    "language": "typescript",
    "fingerprint": {
      "tagName": "h1",
      "textContent": "Example Domain"
    }
  }' | jq .
```

> **Verify:** Returns healed selector (e.g., text match strategy).

Step 6: Report Success

```
curl -s -X POST http://localhost:3700/api/heal/report \
  -H "Content-Type: application/json" \
  -d '{
    "selectorKey": "css:h1.old-heading",
    "url": "https://example.com",
    "healed": true,
    "strategy": "textMatch",
    "confidence": 0.75,
    "framework": "playwright",
    "originalSelector": "h1.old-heading",
    "healedSelector": "text=Example Domain",
    "durationMs": 30
  }'
```

Step 7: Verify Full Flow in UI

1. **Dashboard** (`/`): HealingCard shows updated stats
2. **Healing** → **Overview:** Trend chart shows today's heal, strategy breakdown shows textMatch
3. **Healing** → **Events:** New event row visible with strategy "textMatch"
4. **Click event row:** Detail panel shows `h1.old-heading` → `text=Example Domain`
5. **Healing** → **Fingerprints:** `css:h1` fingerprint shows success count
6. **Settings** → **Self-Healing:** Config values match your settings

Step 8: Clean Up

```
# In qabot REPL:
/heal remove /tmp/e2e-heal-test
```

```
rm -rf /tmp/e2e-heal-test
```

Quick Reference Tables

All API Endpoints

Method	Path	Purpose
POST	`/api/heal/resolve`	Main healing — receive broken selector, return healed
POST	`/api/heal/fingerprint`	Store element baseline fingerprint
POST	`/api/heal/report`	Report healing success/failure
GET	`/api/heal/events`	Query healing event log
GET	`/api/heal/analytics`	Rich analytics (charts, breakdowns)
GET	`/api/heal/injections`	List injected projects
POST	`/api/heal/inject`	Record injection metadata
DELETE	`/api/heal/injections/:id`	Remove injection
GET	`/api/heal/adapters`	List supported adapters
GET	`/api/healing/stats`	Basic healing stats
GET	`/api/healing/fingerprints`	Browse/search stored fingerprints
DELETE	`/api/healing/fingerprints/:id`	Delete a fingerprint
GET	`/api/healing/config`	Read healing configuration
PUT	`/api/healing/config`	Update healing configuration
GET	`/api/healing/by-url`	Fingerprints for a specific URL

All UI Routes

Route	Page	What to Test
`/`	Dashboard	HealingCard in bottom section

`/healing`	Healing Dashboard	5 tabs: Overview, Events, Fingerprints, Projects, Adapters
`/settings`	Settings	Self-Healing section with config controls

All CLI Commands

Command	What it Does
`/heal`	Show healing status (alias for `/heal status`)
`/heal status`	Detailed status with stats
`/heal inject [path]`	Inject healing into test project
`/heal report [days]`	Generate analytics report
`/heal remove <path>`	Remove healing from project
`/heal adapters`	List supported frameworks

6-Strategy Healing Chain

#	Strategy	Description	Confidence
1	Fingerprint Match	Match by stored element attributes (tag, id, class, text, role)	0.85–0.92
2	Similar Selector	Find CSS selector with high textual similarity	0.70–0.85
3	Text Match	Locate element by visible text content	0.75
4	Position Match	Find element at similar bounding box position	0.60–0.70
5	Ancestor Search	Navigate parent/sibling tree to locate element	0.60–0.75
6	AI Healing	LLM analyzes fingerprint + DOM to suggest selector	0.60+

Supported Adapters (6 Framework/Language Combinations)

Framework	Language	Generated Files

Playwright	TypeScript/JS	`tests/helpers/healing-client.ts`, `tests/fixtures/healing.ts`
Selenium	Java	5 files in `src/test/java/healing/`
Selenium	Python	6 files in `tests/healing/`
Cypress	TypeScript	4 files in `cypress/support/` + `cypress/plugins/`
WebdriverIO	TypeScript	3 files in `test/helpers/`
Selenium	C#/.NET	5 files in `Healing/`